跌倒检测

跌倒检测

- 例程实验效果
  - 代码解析
  - 流程图
- 摔倒检测算法简述

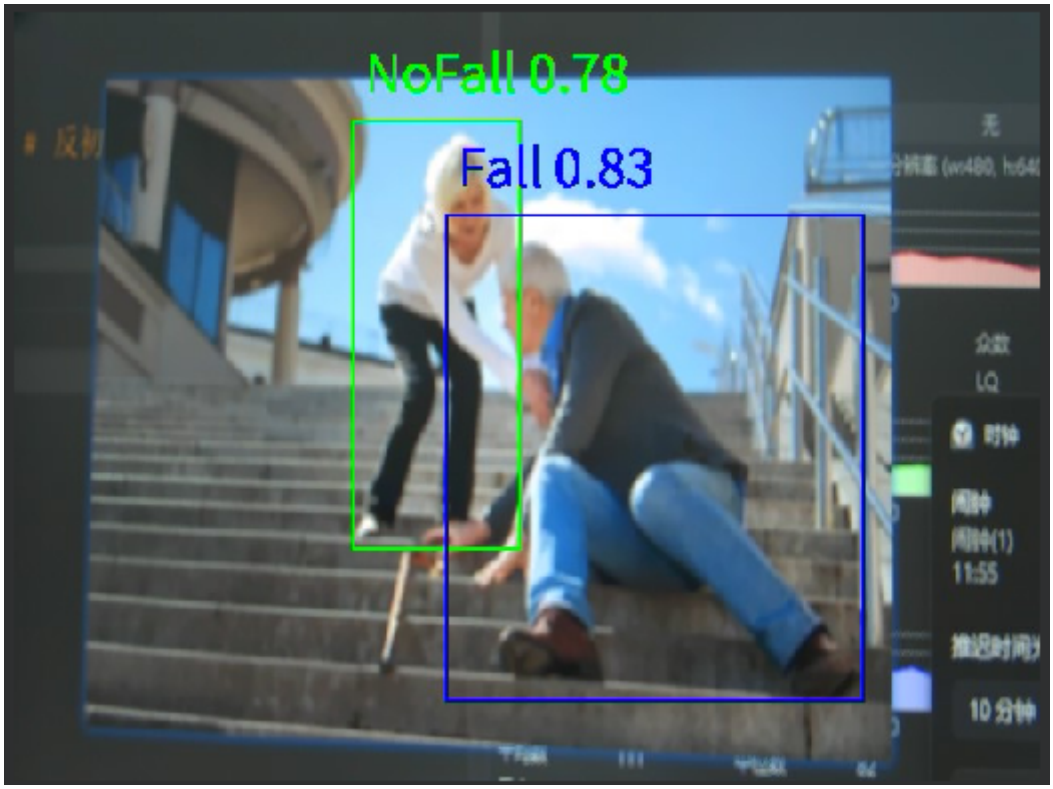
例程实验效果

> 跌倒检测是我们上一节学习的人体关键点检测的一个实际应用场景

本节我们来学习一下使用K230实现跌倒检测的功能。

例程代码在【源码汇总 / 08.Body / 04.falldown_detect.py 】

例程运行效果如下，K230会识别出画面中人物的状态（跌倒 / 没有跌倒），并给出相应的置信度



> 已添加串口输出功能
>
> 检测到人体后会发送如下格式的串口输出
>
> $x,y,w,h,是否跌倒,置信度#
>
> 其中'$'代表数据的开头, '#'代表数据的结尾
>
> x，y，w，h是人体检测框的位置（分辨率为640*480）
>
> 我们举几个例子说明一下：
>
> 如果串口输出：$489,0,146,253,NoFall,0.38#

> 说明在(489,0,146,253)的位置检测到了一个人
>
> NoFall说明没有跌倒
>
> 0.38为置信度，置信度取值范围在0~1之间，越高说明检查正确的可能性越大
>
> 如果串口输出：$483,60,156,386,Fall,0.35#
>
> 说明在(483,60,156,386)的位置检测到了一个人
>
> Fall说明跌倒了
>
> 0.35为置信度，置信度取值范围在0~1之间，越高说明检查正确的可能性越大

代码解析

完整代码请参考【源码汇总 / 08.Body / 02.person_keypoint_detect.py】文件

```
if __name__ == "__main__":

    display_mode="lcd"
    # 设置RGB888P图像大小
    # Set RGB888P image size
    rgb888p_size = [640,360]

    # 根据显示模式设置显示分辨率
    # Set display resolution according to display mode
    if display_mode=="hdmi":
        display_size=[1920,1080]
    else:
        display_size=[640,480]

    # 设置模型路径和其他参数
    # Set model path and other parameters
    kmodel_path = "/sdcard/kmodel/yolov5n-falldown.kmodel"  # 模型文件路径 / Model
file path
    confidence_threshold = 0.3  # 置信度阈值 / Confidence threshold
    nms_threshold = 0.45  # NMS阈值 / NMS threshold
    labels = ["Fall","NoFall"]  # 模型输出类别名称 / Model output category names

    # anchor设置，用于YOLOv5检测框解码
    # Anchor settings for YOLOv5 detection box decoding
    anchors = [10, 13, 16, 30, 33, 23, 30, 61, 62, 45, 59, 119, 116, 90, 156,
198, 373, 326]

    # 初始化PipeLine，用于图像处理流程
    # Initialize PipeLine for image processing workflow
    pl = PipeLine(rgb888p_size=rgb888p_size, display_size=display_size,
display_mode=display_mode)
    pl.create()

    # 初始化自定义跌倒检测实例
    # Initialize custom fall detection instance
    fall_det = FallDetectionApp(
        kmodel_path,               # 模型路径 / Model path
        model_input_size=[640, 640],  # 模型输入大小 / Model input size
        labels=labels,             # 标签列表 / Label list
```

```
        anchors=anchors,           # 锚点设置 / Anchor settings
        confidence_threshold=confidence_threshold,  # 置信度阈值 / Confidence
threshold
        nms_threshold=nms_threshold,  # NMS阈值 / NMS threshold
        nms_option=False,          # NMS选项 / NMS option
        strides=[8,16,32],         # 步长设置 / Stride settings
        rgb888p_size=rgb888p_size, # RGB图像大小 / RGB image size
        display_size=display_size,  # 显示大小 / Display size
        debug_mode=0               # 调试模式 / Debug mode
    )

    # 配置预处理流程
    # Configure preprocessing pipeline
    fall_det.config_preprocess()

    # 主循环，持续检测并显示结果
    # Main loop, continuously detect and display results
    while True:
        with ScopedTiming("total",1):
            # 获取当前帧数据
            # Get current frame data
            img = pl.get_frame()

            # 推理当前帧
            # Infer current frame
            res = fall_det.run(img)

            # 绘制结果到PipeLine的osd图像
            # Draw results to PipeLine's osd image
            fall_det.draw_result(pl, res)

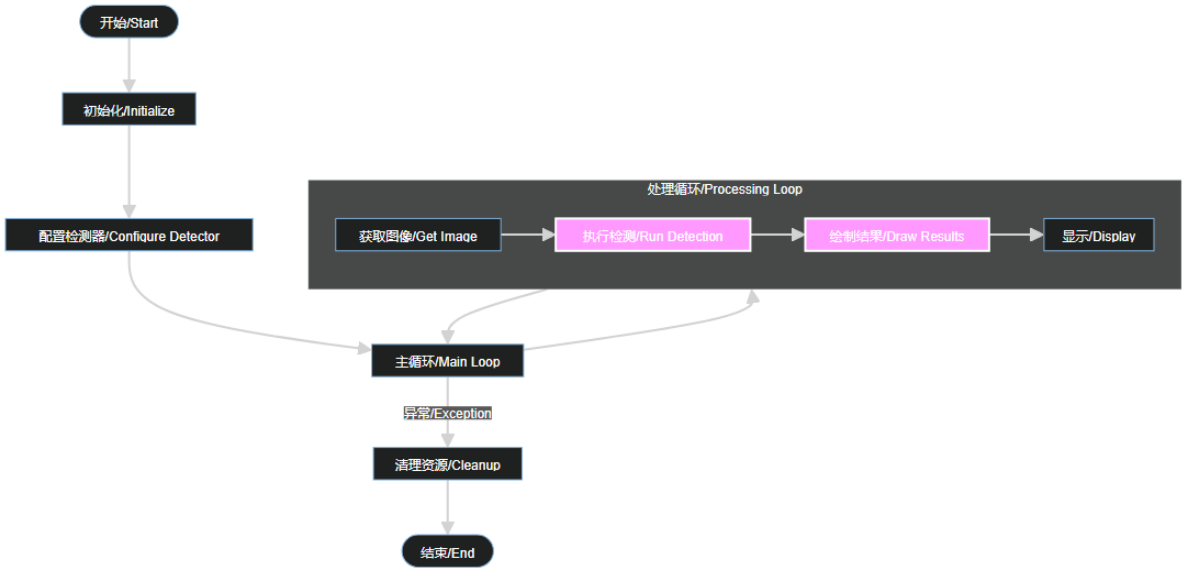
            # 显示当前的绘制结果
            # Display current drawing results
            pl.show_image()

            # 垃圾回收，释放内存
            # Garbage collection to free memory
            gc.collect()

    # 反初始化跌倒检测实例
    # Deinitialize fall detection instance
    fall_det.deinit()

    # 销毁PipeLine实例
    # Destroy PipeLine instance
    pl.destroy()
```

流程图



摔倒检测算法简述

这是一个基于 YOLOv5 的跌倒检测实现：

1. 整体架构
   代码实现了一个跌倒检测应用 (FallDetectionApp)，继承自AIBase基类。主要包含以下功能模块:

- 图像预处理
- 模型推理
- 后处理
- 结果可视化

2. 核心流程

```
图像输入 -> 预处理(AI2D) -> 模型推理 -> 后处理 -> 结果绘制
```

3. 关键步骤详解

(1) 预处理

- 使用AI2D进行图像预处理,包括:
  - padding: 填充边缘使图像符合模型输入要求
  - resize: 将图像缩放到模型输入大小(640x640)
- 预处理保持图像比例不变,避免形变

(2) 模型推理

- 使用YOLOv5-n作为基础网络
- 模型输入为640x640的RGB图像
- 输出包含3个检测头,对应3种不同尺度的检测

(3) 后处理

- 使用anchor-based检测后处理
- 包含以下步骤:
  - 检测框解码
  - 置信度过滤(threshold=0.3)
  - NMS处理(threshold=0.45)

- 输出检测框的类别、置信度和坐标信息

(4) 可视化

- 将检测结果映射回显示分辨率
- 绘制边界框和类别标签
- 支持两个类别:"Fall"(跌倒)和"NoFall"(未跌倒)